

Computational Thinking dengan Dart

Dosen	: IKetut Gunawan, S.KOM, M.T.I
Kode/Mata Kuliah	: KB1185/ Aplikasi Mobile
Bobot Mata Kuliah	: 3 Sks
Prodi	: Teknik Informatika
Pertemuan	: 2

Cakupan Materi

- Computational Thinking: Decomposition, Pattern Recognition, Abstraction, Algorithm
- Flowchart, Pseudocode & Requirement (Business Rules)
- Dart untuk implementasi: Operator & Ekspresi
- Percabangan (**if**, **switch**) dan Perulangan (**for**, **while**)
- Function sebagai hasil Decomposition
- Operasi Collection (**where**, **map**, **fold**, **any**) sebagai Pattern
- Class & Enum sebagai Abstraction
- Studi Kasus: Check-in Absensi dari Requirement sampai Kode Dart
- Assignment: HW 2 — Computational Thinking dengan Dart

Tujuan Pembelajaran

Pada Pertemuan 1 kita sudah mempelajari dasar Dart: tipe data, null safety, `final/const/late`, dan collection. Pertemuan ini menjawab pertanyaan berikutnya: **bagaimana mengubah sebuah masalah nyata menjadi program Dart yang benar?**

Setelah mengikuti pertemuan ini, mahasiswa diharapkan mampu:

1. Menjelaskan empat pilar computational thinking.
2. Menganalisis requirement dan business rule dari sebuah use case.
3. Menyusun algoritma dalam bentuk flowchart dan pseudocode.
4. Menggunakan operator, percabangan, dan perulangan di Dart.
5. Memecah solusi menjadi beberapa function (decomposition).
6. Mengenali pola pengolahan data dan menerapkannya dengan operasi collection.
7. Membuat class dan enum sederhana sebagai abstraction.
8. Menerjemahkan pseudocode menjadi program Dart yang dapat diuji.

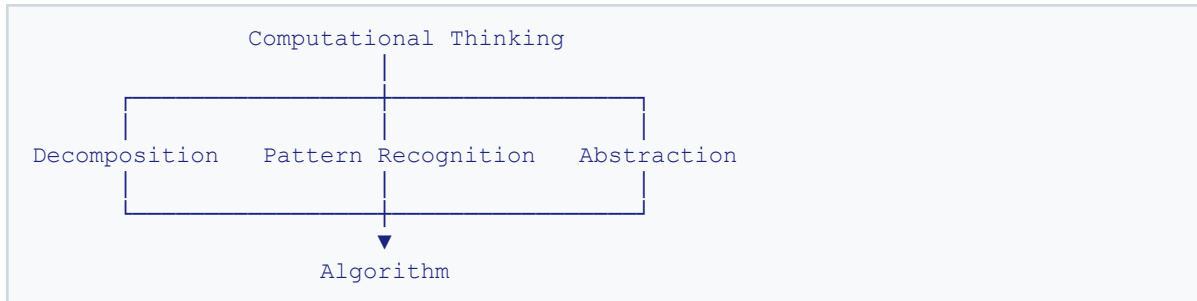
Rencana Aktivitas Kelas (150 menit)

Waktu	Aktivitas
15 menit	Review Dart Pertemuan 1 & pengantar Computational Thinking
20 menit	Empat pilar CT, flowchart, pseudocode, business rule
15 menit	Operator & percabangan
15 menit	Perulangan
15 menit	Function (decomposition)
15 menit	Operasi collection (pattern)
10 menit	Class & enum (abstraction)
20 menit	Live coding studi kasus Check-in Absensi
20 menit	Latihan di kelas
5 menit	Penjelasan assignment

1

Computational Thinking

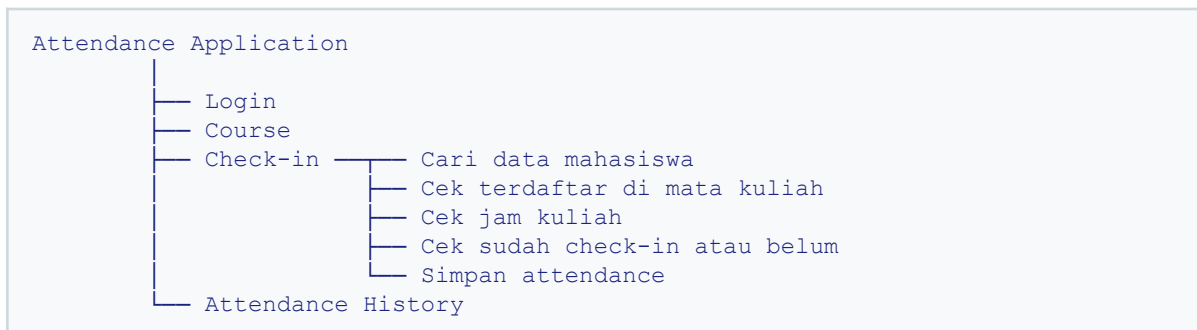
Computational thinking adalah cara berpikir untuk menyelesaikan masalah secara **terstruktur, logis, dan dapat diimplementasikan oleh komputer**. Empat pilar utamanya:



Pilar	Arti	Wujudnya di Dart
Decomposition	Memecah masalah besar menjadi bagian kecil.	Function — satu function untuk satu tugas
Pattern Recognition	Menemukan pola yang berulang dari beberapa masalah.	Perulangan, function yang dipakai ulang, <i>where / map / fold</i>
Abstraction	Mengambil yang penting, mengabaikan detail yang tidak perlu.	Class, enum, parameter function
Algorithm	Langkah-langkah terstruktur untuk menyelesaikan masalah.	Urutan statement, <i>if, switch, for, while</i>

1.1. Decomposition

Masalah "*Buat aplikasi absensi mahasiswa*" terlalu besar jika langsung dikerjakan. Kita pecah:



Setiap bagian paling bawah nantinya menjadi **satu function** di Dart.

1.2. Pattern Recognition

Banyak proses memiliki pola yang sama. Check-in, pinjam buku, bayar parkir, dan setor sampah semuanya mengikuti pola:



Pola pengolahan data juga berulang: **menyaring** data (filter), **mengubah** data (transform), **menjumlahkan** data (aggregate), dan **mencari** data (find). Di Dart, pola ini punya bentuk kode yang tetap (lihat Bab 6).

1.3. Abstraction

Data mahasiswa di dunia nyata sangat banyak: NIM, nama, alamat, tanggal lahir, hobi, dan lain-lain. Untuk sistem absensi kita hanya membutuhkan:

```
Student
├── id
├── name
└── courses
```

Hasil abstraction inilah yang nantinya ditulis sebagai **class** di Dart.

1.4. Algorithm, Flowchart, dan Pseudocode

Algoritma dapat digambarkan dengan **flowchart** (visual) atau ditulis dengan **pseudocode** (teks yang belum terikat syntax bahasa tertentu).

Simbol Flowchart	Fungsi	Padanan di Dart
Oval	Start / End	<code>void main() { ... }</code>
Rectangle	Process	Assignment, pemanggilan function
Diamond	Decision	<code>if / switch</code>
Parallelogram	Input / Output	Parameter / <code>print()</code>
Arrow	Flow	Urutan eksekusi statement

1.5. Requirement dan Business Rule

Sebelum menulis algoritma, pahami dulu kebutuhannya:

- **Functional requirement** — apa yang harus dilakukan sistem. Contoh: *User dapat melakukan check-in.*
- **Non-functional requirement** — bagaimana sistem harus bekerja. Contoh: *Password tidak disimpan dalam plaintext.*
- **Business rule** — aturan bisnis yang wajib dipatuhi. Contoh: *Mahasiswa hanya boleh check-in satu kali per sesi.*

Setiap **business rule** nantinya menjadi **satu validasi** di kode (biasanya berupa `if`). Jika business rule belum jelas, kode yang dihasilkan bisa benar secara syntax tetapi salah secara kebutuhan.

1.6. Alur dari Masalah ke Kode

```
PROBLEM → REQUIREMENT & BUSINESS RULE → DECOMPOSITION  
→ ABSTRACTION → ALGORITHM (FLOWCHART / PSEUDOCODE)  
→ DART CODE → TEST
```

Hindari kebiasaan: *"Buka DartPad → langsung coding → coba-coba sampai jalan."*
Kode adalah tahap **implementasi**, bukan tahap berpikir.

2

Operator & Ekspresi

Operator digunakan untuk menghitung dan membandingkan nilai. Hampir setiap business rule membutuhkan operator.

Jenis	Operator	Contoh	Hasil
Aritmatika	+ - * /	10 / 4	2.5 (selalu double)
Bagi bulat	~/	17 ~/ 5	3
Sisa bagi	%	17 % 5	2
Perbandingan	== != > < >= <=	nilai >= 60	true / false
Logika	&& (dan), (atau), ! (tidak)	lulus && hadirCukup	true / false
Penugasan	= += -= ++ --	total += harga	total bertambah
Null-aware	??	catatan ?? '-'	nilai pengganti jika null

Contoh: menghitung nilai akhir

```
void main() {
    int nilaiTugas = 80;
    int nilaiUts = 70;
    int nilaiUas = 90;
    int kehadiran = 12; // dari 14 pertemuan

    double nilaiAkhir = nilaiTugas * 0.3 + nilaiUts * 0.3 + nilaiUas * 0.4;
    bool nilaiCukup = nilaiAkhir >= 60;
    bool hadirCukup = kehadiran >= 11;

    print('Nilai akhir: ${nilaiAkhir.toStringAsFixed(1)}'); // 81.0
    print('Lulus: ${nilaiCukup && hadirCukup}');           // true
    print('17 ~/ 5 = ${17 ~/ 5}');                          // 3
    print('17 % 5 = ${17 % 5}');                            // 2
}
```

Gunakan `toStringAsFixed(n)` untuk menampilkan angka desimal dengan `n` digit di belakang koma. Operator `~/` dan `%` sering dipakai untuk konversi waktu, misalnya `150 ~/ 60` jam dan `150 % 60` menit.

3

Percabangan

Percabangan adalah wujud simbol **Decision (diamond)** pada flowchart dan **IF ... THEN ... ELSE** pada pseudocode.

3.1. if, else if, else

```
String tentukanGrade(double nilai) {  
    if (nilai >= 85) {  
        return 'A';  
    } else if (nilai >= 70) {  
        return 'B';  
    } else if (nilai >= 60) {  
        return 'C';  
    } else {  
        return 'D';  
    }  
}  
  
void main() {  
    print(tentukanGrade(90)); // A  
    print(tentukanGrade(72)); // B  
    print(tentukanGrade(40)); // D  
}
```

Urutan kondisi penting. Jika **nilai >= 60** diperiksa lebih dulu, nilai 90 akan mendapat grade C.

3.2. Guard Clause (Early Return)

Untuk memeriksa banyak business rule, periksa kondisi **gagal** lebih dulu lalu langsung **return**. Kode menjadi datar dan mudah dibaca:

```
String pinjamBuku(int jumlahDipinjam, bool bukuTersedia) {  
    if (jumlahDipinjam >= 3) {  
        return 'Gagal: maksimal pinjam 3 buku'; // BR-01  
    }  
    if (!bukuTersedia) {  
        return 'Gagal: buku sedang dipinjam'; // BR-02  
    }  
    return 'Berhasil meminjam buku';  
}
```

3.3. Operator Ternary

Untuk percabangan sederhana yang menghasilkan nilai:

```
int umur = 20;  
String status = umur >= 17 ? 'Dewasa' : 'Anak-anak';
```

3.4. switch

Gunakan **switch** jika memeriksa satu nilai terhadap banyak kemungkinan. Di Dart 3 tidak perlu **break**:

```
String hari = 'Sabtu';  
  
switch (hari) {  
  case 'Sabtu':  
  case 'Minggu':  
    print('Libur');  
  default:  
    print('Hari kuliah');  
}
```

Switch expression menghasilkan nilai secara langsung (**_** berarti "selain itu"):

```
String grade = 'B';  
  
String keterangan = switch (grade) {  
  'A' => 'Sangat baik',  
  'B' => 'Baik',  
  'C' => 'Cukup',  
  _ => 'Perlu perbaikan',  
};
```

4

Perulangan

Perulangan adalah wujud **pattern recognition**: langkah yang sama dijalankan berulang kali untuk data yang berbeda.

Bentuk	Kapan digunakan
<code>`for (int i = 0; i < n; i++)`</code>	Jumlah perulangan diketahui, atau butuh indeks.
<code>`for (final item in list)`</code>	Memproses setiap elemen List/Set.
<code>`while (kondisi)`</code>	Jumlah perulangan belum diketahui, berhenti saat kondisi false.
<code>`do { } while (kondisi)`</code>	Sama dengan while, tetapi dijalankan minimal satu kali.

4.1. for dan for-in

```
void main() {  
    // for dengan indeks  
    for (int i = 1; i <= 3; i++) {  
        print('Pertemuan ke-$i');  
    }  
  
    // for-in: menjumlahkan total belanja  
    List<int> belanja = [15000, 20000, 5000];  
    int total = 0;  
    for (final harga in belanja) {  
        total += harga;  
    }  
    print('Total: $total'); // Total: 40000  
}
```

4.2. while

Contoh: PIN diberi kesempatan maksimal 3 kali.

```
void main() {  
    List<String> inputPin = ['1111', '2222', '1234'];  
    int percobaan = 0;  
    bool berhasil = false;  
  
    while (!berhasil && percobaan < 3) {  
        String pin = inputPin[percobaan];  
        percobaan++;  
        if (pin == '1234') {  
            berhasil = true;  
        }  
    }  
  
    print('Berhasil: $berhasil, percobaan: $percobaan');  
    // Berhasil: true, percobaan: 3  
}
```

4.3. break dan continue

```
void main() {  
    List<int> nilai = [70, 0, 85, 40, 90];  
  
    for (final n in nilai) {  
        if (n == 0) {  
            continue; // lewati data kosong, lanjut ke elemen berikutnya  
        }  
        if (n < 50) {  
            print('Ditemukan nilai di bawah 50: $n');  
            break; // hentikan perulangan  
        }  
        print('Nilai aman: $n');  
    }  
}
```

5

Function

Function adalah wujud **decomposition** di kode. Setiap bagian hasil pemecahan masalah menjadi satu function yang **melakukan satu tugas**.

5.1. Anatomi Function

```
// tipe return    nama           parameter
double          hitungLuas    (double panjang, double lebar) {
    return panjang * lebar;    // nilai yang dikembalikan
}
```

Bentuk	Contoh
Tanpa nilai kembali	<code>void tampilkan(String pesan) { print(pesan); }</code>
Dengan nilai kembali	<code>int kuadrat(int x) { return x * x; }</code>
Arrow function	<code>int kuadrat(int x) => x * x;</code>
Named parameter	<code>double bayar({required double total, double diskon = 0})</code>
Nullable return	<code>Student? cariStudent(String id)</code>

5.2. Positional vs Named Parameter

```
// positional: urutan argumen harus sesuai
double hitungDiskon(double total, double persen) {
    return total * persen / 100;
}

// named: argumen ditulis dengan nama, lebih jelas dibaca
double hitungTotalBayar({required double total, double persen = 0}) {
    return total - hitungDiskon(total, persen);
}

void main() {
    print(hitungDiskon(200000, 10)); // 20000.0
    print(hitungTotalBayar(total: 200000, persen: 10)); // 180000.0
    print(hitungTotalBayar(total: 50000)); // 50000.0
}
```

5.3. Contoh Decomposition menjadi Function

Masalah: *"Hitung nilai akhir, grade, dan status kelulusan mahasiswa."*

```
double hitungNilaiAkhir(int tugas, int uts, int uas) {  
    return tugas * 0.3 + uts * 0.3 + uas * 0.4;  
}  
  
bool cekKehadiran(int hadir, int totalPertemuan) {  
    return hadir / totalPertemuan >= 0.75;  
}  
  
String tentukanStatus(double nilai, bool hadirCukup) {  
    if (!hadirCukup) return 'Tidak lulus (kehadiran kurang)';  
    if (nilai < 60) return 'Tidak lulus (nilai kurang)';  
    return 'Lulus';  
}  
  
void main() {  
    double nilai = hitungNilaiAkhir(80, 70, 90);  
    bool hadir = cekKehadiran(12, 14);  
    print(tentukanStatus(nilai, hadir)); // Lulus  
}
```

Ciri function yang baik: namanya berupa kata kerja yang jelas, melakukan **satu** tugas, dan hasilnya hanya ditentukan oleh parameternya sehingga **mudah diuji**.

6

Operasi Collection

Pada Pertemuan 1 kita sudah mengenal **List**, **Set**, dan **Map**. Sekarang kita memakai operasinya untuk pola pengolahan data yang sering muncul (**pattern recognition**).

Pola	Pertanyaan	Operasi Dart
Filter	Data mana yang memenuhi syarat?	<code>where(...).toList()</code>
Transform	Ubah setiap data menjadi bentuk lain.	<code>map(...).toList()</code>
Aggregate	Berapa total / jumlahnya?	<code>fold(...), length</code>
Check	Adakah / apakah semuanya memenuhi syarat?	<code>any(...), every(...), contains(...)</code>
Find	Di mana data tertentu?	<code>for-in + return, indexWhere(...)</code>

6.1. Operasi pada List

```
void main() {
  List<int> nilai = [90, 55, 78, 62, 40];

  List<int> lulus = nilai.where((n) => n >= 60).toList();
  print(lulus); // [90, 78, 62]

  List<String> label = nilai.map((n) => n >= 60 ? 'L' : 'TL').toList();
  print(label); // [L, TL, L, L, TL]

  int total = nilai.fold(0, (sum, n) => sum + n);
  print(total); // 325

  print(nilai.any((n) => n < 60)); // true
  print(nilai.every((n) => n >= 60)); // false
  print(nilai.contains(78)); // true
  print(nilai.indexWhere((n) => n < 60)); // 1
}
```

`(n) => n >= 60` adalah **anonymous function** (function tanpa nama). Artinya: "untuk setiap `n`, kembalikan hasil `n >= 60`".

6.2. Operasi pada Map

```
void main() {  
    Map<String, int> stok = {'Pulpen': 10, 'Buku': 5};  
  
    stok['Pensil'] = 7;                // tambah data  
    stok['Buku'] = stok['Buku']! - 1; // ubah data  
    print(stok.containsKey('Penghapus')); // false  
  
    for (final item in stok.entries) {  
        print('${item.key}: ${item.value}');  
    }  
    // Pulpen: 10  
    // Buku: 4  
    // Pensil: 7  
}
```

7

Class & Enum

Class dan enum adalah wujud **abstraction** di kode: kita hanya menuliskan atribut dan perilaku yang penting untuk masalah yang sedang diselesaikan.

7.1. Enum

Enum digunakan untuk nilai yang pilihannya **terbatas dan sudah pasti**, misalnya status. Lebih aman daripada String karena salah ketik langsung terdeteksi oleh compiler.

```
enum StatusBuku { tersedia, dipinjam }
```

7.2. Class Sederhana

```
enum StatusBuku { tersedia, dipinjam }

class Buku {
    final String kode;
    final String judul;
    StatusBuku status;

    Buku({
        required this.kode,
        required this.judul,
        this.status = StatusBuku.tersedia,
    });

    bool get bisaDipinjam => status == StatusBuku.tersedia;

    void pinjam() {
        status = StatusBuku.dipinjam;
    }
}

void main() {
    final buku = Buku(kode: 'B01', judul: 'Dasar Dart');

    print(buku.bisaDipinjam); // true
    buku.pinjam();
    print(buku.bisaDipinjam); // false
    print(buku.status.name); // dipinjam
}
```

Bagian	Penjelasan
`final String kode`	Atribut yang tidak berubah setelah object dibuat.
`StatusBuku status`	Atribut yang dapat berubah (tidak final).

Bagian	Penjelasan
<code>`Buku({required this.kode, ...})`</code>	Constructor dengan named parameter.
<code>`bool get bisaDipinjam`</code>	Getter: nilai turunan yang dihitung dari atribut.
<code>`void pinjam()`</code>	Method: perilaku yang dimiliki object.

Gunakan **class** dibanding `Map<String, dynamic>` untuk data utama aplikasi: nama atribut dan tipenya diperiksa oleh compiler, sehingga kesalahan ditemukan lebih awal.

8

Dari Pseudocode ke Dart

Setelah pseudocode benar, penerjemahan ke Dart hampir satu-banding-satu:

Pseudocode	Dart
INPUT x	Parameter function atau variabel
DISPLAY "teks"	<code>print('teks');</code>
IF kondisi THEN ... ELSE ... END IF	<code>if (kondisi) { ... } else { ... }</code>
FOR EACH item IN daftar	<code>for (final item in daftar) { ... }</code>
WHILE kondisi DO	<code>while (kondisi) { ... }</code>
FIND data WHERE syarat	<code>where(...), any(...), atau for-in + return</code>
SAVE data	<code>daftar.add(data);</code>
STOP / RETURN hasil	<code>return hasil;</code>
PROCEDURE namaProses	Function: <code>TipeHasil namaProses(...) { ... }</code>
Data / entitas	<code>class</code> dan <code>enum</code>

Setiap langkah pseudocode sebaiknya dapat ditunjuk di kode. Jika ada kode yang tidak ada di pseudocode (atau sebaliknya), berarti analisis atau implementasinya belum sinkron.

9

Studi Kasus: Check-in Absensi

Studi kasus ini menunjukkan seluruh alur: dari requirement sampai kode Dart yang diuji.

Problem statement: Kampus membutuhkan fitur check-in absensi. Mahasiswa hanya boleh check-in untuk mata kuliah yang diambil, pada jam kuliah, dan satu kali per hari untuk setiap mata kuliah.

9.1. Requirement & Business Rule

Kode	Isi
FR-01	Mahasiswa dapat melakukan check-in untuk sebuah mata kuliah.
FR-02	Sistem menampilkan hasil check-in (berhasil atau alasan gagal).
BR-01	Mahasiswa harus terdaftar dan mengambil mata kuliah tersebut.
BR-02	Check-in hanya dapat dilakukan pada jam kuliah 08:00–10:30.
BR-03	Mahasiswa hanya boleh check-in satu kali per hari per mata kuliah.

9.2. Input, Output, Abstraction

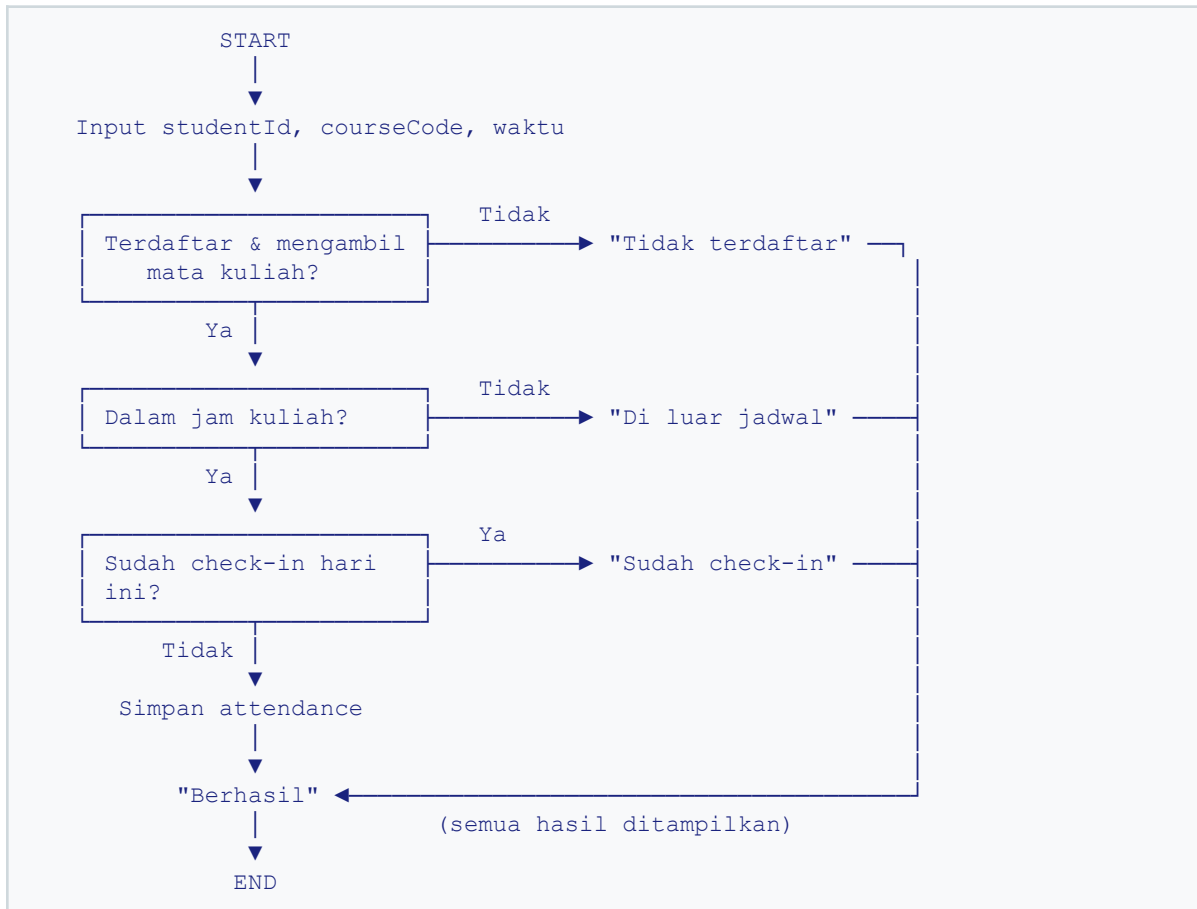
Aspek	Hasil Analisis
Input	ID mahasiswa, kode mata kuliah, waktu check-in
Output	Status check-in: berhasil / tidak terdaftar / di luar jadwal / sudah check-in
Abstraction	<code>Student</code> (id, name, courses), <code>Attendance</code> (studentId, courseCode, date), <code>enum CheckInStatus</code>

9.3. Decomposition

```

checkIn
├── findStudent      → cari mahasiswa berdasarkan ID      (BR-01)
├── isEnrolled       → cek mahasiswa mengambil mata kuliah (BR-01)
├── isWithinSchedule → cek jam check-in                  (BR-02)
├── hasCheckedIn     → cek sudah check-in hari ini        (BR-03)
└── simpan attendance
```

9.4. Flowchart



9.5. Pseudocode

```

PROCEDURE checkIn(studentId, courseCode, waktu)
  student = FIND student WHERE id = studentId
  IF student NOT FOUND OR courseCode NOT IN student.courses THEN
    RETURN "Tidak terdaftar"
  END IF
  IF waktu NOT BETWEEN 08:00 AND 10:30 THEN
    RETURN "Di luar jadwal"
  END IF
  IF attendance EXISTS WHERE studentId, courseCode, tanggal THEN
    RETURN "Sudah check-in"
  END IF
  SAVE attendance(studentId, courseCode, waktu)
  RETURN "Berhasil"
END PROCEDURE
  
```

9.6. Implementasi Dart

Salin kode berikut ke DartPad dan jalankan. Perhatikan komentar yang menunjukkan hubungan kode dengan hasil analisis.

```
// ----- ABSTRACTION -----
enum CheckInStatus {
    success,
    notEnrolled,
    outsideSchedule,
    alreadyCheckedIn,
}

class Student {
    final String id;
    final String name;
    final List<String> courses;

    Student(this.id, this.name, this.courses);
}

class Attendance {
    final String studentId;
    final String courseCode;
    final DateTime date;

    Attendance(this.studentId, this.courseCode, this.date);
}

// ----- DATA -----
final List<Student> students = [
    Student('S01', 'Budi', ['KB1185', 'KB1101']),
    Student('S02', 'Citra', ['KB1185']),
];

final List<Attendance> attendances = [];

// ----- DECOMPOSITION -----
Student? findStudent(String id) {
    for (final student in students) {
        if (student.id == id) return student;
    }
    return null;
}

bool isEnrolled(Student student, String courseCode) {
    return student.courses.contains(courseCode);
}

bool isWithinSchedule(DateTime time) {
    final minutes = time.hour * 60 + time.minute;
    return minutes >= 8 * 60 && minutes <= 10 * 60 + 30; // 08:00-10:30
}

bool isSameDay(DateTime a, DateTime b) {
    return a.year == b.year && a.month == b.month && a.day == b.day;
}

bool hasCheckedIn(String studentId, String courseCode, DateTime time) {
    return attendances.any((a) =>
        a.studentId == studentId &&
        a.courseCode == courseCode &&
        isSameDay(a.date, time));
}
```

```
}

// ----- ALGORITHM -----
CheckInStatus checkIn(String studentId, String courseCode, DateTime time) {
    final student = findStudent(studentId);
    if (student == null || !isEnrolled(student, courseCode)) {
        return CheckInStatus.notEnrolled; // BR-01
    }
    if (!isWithinSchedule(time)) {
        return CheckInStatus.outsideSchedule; // BR-02
    }
    if (hasCheckedIn(studentId, courseCode, time)) {
        return CheckInStatus.alreadyCheckedIn; // BR-03
    }
    attendances.add(Attendance(studentId, courseCode, time));
    return CheckInStatus.success;
}

String toMessage(CheckInStatus status) => switch (status) {
    CheckInStatus.success => 'Check-in berhasil',
    CheckInStatus.notEnrolled => 'Gagal: tidak terdaftar',
    CheckInStatus.outsideSchedule => 'Gagal: di luar jam kuliah',
    CheckInStatus.alreadyCheckedIn => 'Gagal: sudah check-in',
};

// ----- TEST SCENARIO -----
void main() {
    final pagi = DateTime(2026, 10, 5, 8, 15);
    final siang = DateTime(2026, 10, 5, 13, 0);

    // Skenario 1 - expected: Check-in berhasil
    print(toMessage(checkIn('S01', 'KB1185', pagi)));

    // Skenario 2 (BR-03) - expected: Gagal: sudah check-in
    print(toMessage(checkIn('S01', 'KB1185', pagi)));

    // Skenario 3 (BR-01) - expected: Gagal: tidak terdaftar
    print(toMessage(checkIn('S02', 'KB1101', pagi)));

    // Skenario 4 (BR-01) - expected: Gagal: tidak terdaftar
    print(toMessage(checkIn('S99', 'KB1185', pagi)));

    // Skenario 5 (BR-02) - expected: Gagal: di luar jam kuliah
    print(toMessage(checkIn('S02', 'KB1185', siang)));

    // expected: Total attendance: 1
    print('Total attendance: ${attendances.length}');
}
```

9.7. Tabel Traceability

Tabel ini membuktikan bahwa setiap business rule benar-benar diimplementasikan dan diuji:

Business Rule	Function	Diuji pada skenario
BR-01 Terdaftar di mata kuliah	<code>findStudent, isEnrolled</code>	S02-KB1101, S99-KB1185
BR-02 Jam kuliah 08:00–10:30	<code>isWithinSchedule</code>	S02-KB1185 pukul 13:00
BR-03 Sekali per hari	<code>hasCheckedIn, isSameDay</code>	S01-KB1185 check-in kedua

10

Latihan di Kelas

Kerjakan secara berkelompok. Untuk setiap latihan, tuliskan dulu **business rule**, **decomposition**, dan **pseudocode** di kertas, baru implementasikan di DartPad.

Latihan 1 — Kasir dengan Diskon

Kode	Business Rule
BR-01	Belanja minimal Rp100.000 mendapat diskon 10%.
BR-02	Member mendapat tambahan diskon 5% (hanya jika BR-01 terpenuhi).
BR-03	Total potongan maksimal Rp25.000.

Petunjuk decomposition: `hitungPersenDiskon`, `hitungPotongan`, `hitungTotalBayar`.

Skenario	Total Belanja	Member	Expected Total Bayar
1	Rp80.000	Tidak	Rp80.000
2	Rp150.000	Tidak	Rp135.000
3	Rp150.000	Ya	Rp127.500
4	Rp300.000	Ya	Rp275.000 (potongan dibatasi Rp25.000)

Latihan 2 — Tarif Parkir

Kode	Business Rule
BR-01	Durasi parkir dihitung per jam, sisa menit dibulatkan ke atas (minimal 1 jam).
BR-02	Motor: Rp2.000 jam pertama, Rp1.000 untuk setiap jam berikutnya.
BR-03	Mobil: Rp5.000 jam pertama, Rp3.000 untuk setiap jam berikutnya.

Petunjuk: gunakan `enum JenisKendaraan { motor, mobil }`, operator `~/` dan `%` untuk durasi, dan `switch` untuk tarif.

Skenario	Kendaraan	Durasi	Expected Tarif
1	Motor	30 menit	Rp2.000
2	Motor	150 menit	Rp4.000
3	Mobil	60 menit	Rp5.000
4	Mobil	181 menit	Rp14.000

11

Assignment

HW 2 — Computational Thinking dengan Dart

Pilih **satu** use case berikut. Setiap use case sudah memiliki business rule minimum; kamu boleh menambahkan rule lain yang masuk akal.

Use Case	Business Rule Minimum
Perpustakaan	Maksimal pinjam 3 buku; buku yang sedang dipinjam tidak bisa dipinjam; denda Rp1.000 per hari keterlambatan.
Laundry	Tarif Rp7.000/kg; berat di bawah 2 kg dihitung 2 kg; layanan express +50%.
Parkir Langganan	Member bulanan gratis; non-member tarif progresif; tiket hilang denda Rp20.000.
Bank Sampah	Harga per kg berbeda per kategori (plastik, kertas, logam); penarikan saldo minimal Rp10.000; saldo tidak boleh minus.
Pembayaran (E-Wallet)	Saldo tidak boleh minus; limit transaksi harian Rp2.000.000; PIN salah 3 kali → akun terblokir.
Patroli Security	Setiap checkpoint wajib discan; checkpoint harus sesuai urutan; scan terlambat lebih dari 15 menit dicatat "terlambat".

Bagian A — Dokumen Analisis

1. Problem Statement
2. Actor
3. Input & Output
4. Functional Requirement
5. Business Rules (diberi kode BR-01, BR-02, ...)
6. Decomposition
7. Pattern Recognition
8. Abstraction (class, atribut, enum)
9. Algorithm
10. Flowchart untuk proses utama
11. Pseudocode

Bagian B — Program Dart

Ditulis dalam **satu file** `main.dart` dan dapat dijalankan di DartPad. Checklist minimum:

- [] Minimal **1 class** model dan **1 enum** (abstraction).
- [] Minimal **4 function** hasil decomposition, masing-masing satu tugas.
- [] Menggunakan `List` dan/atau `Map` untuk menyimpan data.
- [] Menggunakan percabangan (`if / switch`) dan perulangan.
- [] **Semua** business rule diimplementasikan.
- [] `main()` berisi minimal **5 skenario uji** (sukses dan gagal) dengan komentar expected output.
- [] Belum menggunakan Flutter.

Bagian C — Tabel Traceability

Buat tabel seperti Bab 9.7: setiap business rule → function yang mengimplementasikan → skenario yang mengujinya.

Kerangka Program

```
// =====  
// HW 2 - <Nama Use Case>  
// Nama : ...  
// NIM  : ...  
// =====  
  
// ----- ABSTRACTION -----  
enum ... { ... }  
  
class ... {  
    // hanya atribut yang penting untuk masalah ini  
}  
  
// ----- DATA -----  
final List<...> ... = [ ... ];  
  
// ----- DECOMPOSITION -----  
// satu function = satu tugas, beri komentar BR yang diimplementasikan  
  
// ----- ALGORITHM -----  
// proses utama, urutannya sesuai pseudocode  
  
// ----- TEST SCENARIO -----  
void main() {  
    // Skenario 1 (sukses)           - expected: ...  
    // Skenario 2 (gagal: BR-01)     - expected: ...  
    // Skenario 3 (gagal: BR-02)     - expected: ...  
}
```

Pengumpulan

- Push ke repository Git yang sudah disiapkan pada assignment Pertemuan 1, di folder [pertemuan-02/](#).
- Isi folder: [analisis.pdf](#) (Bagian A & C), [main.dart](#) (Bagian B), dan [README.md](#) berisi nama use case serta link DartPad.
- Gunakan commit message yang jelas, misalnya [feat: implement business rule BR-02 laundry express](#).
- Analisis dan kode **dikerjakan sendiri terlebih dahulu**.
- Jika ada perubahan jelaskan di [README.md](#) apa yang diubah dan mengapa.

Rubrik Penilaian

Komponen	Bobot
Requirement & business rule jelas dan lengkap	20%
Decomposition, pattern recognition, abstraction	20%
Algorithm, flowchart, pseudocode konsisten	20%
Implementasi Dart sesuai business rule	25%
Skenario uji, traceability, dan kerapian kode	15%
Total	100%

12

Learning Check

Jawab pertanyaan berikut sebelum kelas selesai:

1. Sebutkan empat pilar computational thinking beserta wujudnya di Dart.
2. Mengapa sebuah business rule biasanya menjadi `if` di dalam kode?
3. Apa perbedaan `for-in` dan `while`? Berikan contoh kapan masing-masing digunakan.
4. Apa ciri function yang baik hasil decomposition?
5. Kapan sebaiknya menggunakan `enum` dibandingkan `String`?
6. Apa kegunaan tabel traceability?

Kunci pemahaman: Kode yang baik tidak dimulai dari keyboard, tetapi dari analisis. Jika setiap business rule dapat ditelusuri ke sebuah function dan sebuah skenario uji, maka program tersebut benar secara kebutuhan, bukan hanya benar secara syntax.